



Slides for P3666R1 — Bit-precise integers

Document number:

P3869R1

Date:

2025-02-11

Audience:

SG6

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:

Jan Schultke <janschultke@gmail.com>

Source:

github.com/Eisenwave/cpp-proposals/blob/master/src/bitint-slides.cow



IMPORTANT: This document has custom controls:

- , : go to the next slide
- , : go to previous slide

Bit-precise integers

P3666R1

Introduction

C23 now has `_BitInt` type for N-bit integers (WG14 [N2763], [N2775]):

```
// 8-bit unsigned integer initialized with value 255.  
// The literal suffix wb is unnecessary in this case.  
unsigned _BitInt(8) x = 0xFFwb;
```

- would be very useful in C++ for > 64-bit computation
- *needs* to be in C++ to call C functions portably!
- efforts to standardize in C++ abandoned (mainly [N1744], [P1889R1] (TS))
- implemented by GCC and Clang; max: `_BitInt(8'388'608)`
- **Now:** figuring out the details

P3666R1 Overview

- **Core design:** copy `_BitInt` from C23 & paste w/ minimal changes:
 - allow `_BitInt(1)` ([N3699] "Integer Sets, v3")
 - change type of `0wb` from `_BitInt(2)` to `_BitInt(1)`
 - allow `enum E : _BitInt(N)` ([N3705], "bit-precise enum")
 - add deduction of `N` from `_BitInt(N)`, other C++ stuff
- **Library design:** copy from C23 & paste unchanged:
 - `<cmath>`, `<stdbit.h>`, `<stdckdint.h>`: copy C semantics
 - lots of extensions to C++ standard library

P3666R1 Points of contention

- ⚡ fundamental type instead of library class template
- ⚡ undefined behavior on signed integer overflow
- ⚡ implicit conversions inherited from C
- ⚡ `BITINT_MAXWIDTH` stays the same
- ⚡ `_BitInt` keyword should exist
- (SG22: unanimous consent to author position on *all*)

Library class template vs fundamental type

This was discussed as [\[P3639R0\]](#).

”

The WG14 delegation to SG22 believes that the C++ type family that deliberately corresponds to `_BitInt` (perhaps via compatibility macros) should be... (Fundamental/Library)

SF	F	N	L	SL
8	1	1	0	0

WG21

SF	F	N	L	SL
4	5	0	0	0

- SG6: no opinion
- EWG: consensus to have a fundamental type

UB on signed integer overflow

- **Problem:** signed integer overflow results in UB
- **Idea:** well-defined for `_BitInt`, e.g. wrapping & erroneous (Rust-style)
- ✓ less UB = more good, could catch bugs
- ✗ inconsistent with C
- ✗ not a general solution (`int` stays unchanged)
- ✗ well-defined (wrapping) behavior not very useful
- ✗ UB is useful for optimization
- **Author position:** problem should be approached more generally

Limiting implicit conversions (1/2)

- **Problem:** lots of bug-prone, questionable implicit conversions
- **Idea:** limit conversions for `_BitInt` (e.g. narrowing)
- **✗** almost impossible to specify without "false positives"
 - many issues due to lack of "untyped literals" like in Rust

```
// Problems with C/C++ interoperable headers:
```

```
unsigned _BitInt(8) bit = 0;
```

```
unsigned _BitInt(3) max3u = -1;
```

```
// Problems with "false rejections" for safe cases:
```

```
bit &= 0xf; // int → unsigned _BitInt(8)
```

Limiting implicit conversions (2/2)

```
template<std::integral T>
T div_ceil(T x, T y) { // performs integer division, rounding to +∞
    // ⚠ Could be mixed-sign comparison:
    bool quotient_positive = (x ^ y) >= 0;
    // ⚠ Could be mixed-sign comparison
    bool adjust = x % y != 0 && quotient_positive;
    // ⚠ Could be mixed-sign addition between int (0 or 1)
    // and unsigned _BitInt(N) "x / y":
    // ⚠ Could be lossy conversion: int → unsigned _BitInt
    return x / y + int(adjust);
}
```

Author position: at most, pick low-hanging fruits (e.g. `_BitInt` → `bool`)

Raising the `BITINT_MAXWIDTH`

- **Problem:** C only guarantees `_BitInt(64)` / `_BitInt(LLONG_WIDTH)`
 - `_BitInt(128)` and wider are not portable
- **Idea:** mandate e.g. `BITINT_MAXWIDTH >= 65'535` (GCC max.)
- **✗** `BITINT_MAXWIDTH` depends on system ABI:
 - e.g. x86_64 psABI: Clang supports `_BitInt(8'388'608)`
 - e.g. Basic C ABI for WASM: Clang supports only `_BitInt(128)`
- **✓** standardizing high `BITINT_MAXWIDTH` pressures ecosystem
- **Author position:** inherit C restrictions, which are still useful

C++ standard library changes (1/2)

- § 5.1. Naming of alias template: `bit_int` and `bit_uint` proposed
- § 5.2. `format`, `to_chars` (function template), `to_string` support
- § 5.3. Preventing ranges `::iota_view` ABI break
- § 5.4. `size_t` and `ptrdiff_t` allowed to be bit-precise
- § 5.5. New `abs` overload (function template)
- § 5.6. bit-precise ints also treated as `double` in `<cmath>`
- § 5.7. no `<random>` support (too much design/wording effort)
- § 5.8. `simd` support added *only* for widths of standard integers

C++ standard library changes (2/2)

- § 5.9. `valarray` support added
- § 5.10. `is_integral` includes bit-precise integers
- § 5.11. `make_signed` and `make_unsigned` changes
- § 5.12. misc. support is either blanket support, or inherited from C
 - `<bit>`, `<numeric>`, etc.
 - more design/wording effort in the long run if support is delayed
- § 5.13. one feature-test macro for entire library (and one for core)
- § 5.14. bit-precise integers passed by value, despite potential large size

Questions

- Agreement with author position?
 - allow `0wb = _BitInt(1)` and `enum E : _BitInt(N)`
 - keep UB on signed integer overflow
 - keep all implicit conversions (at least in this paper)
 - keep the `BITINT_MAXWIDTH` from C
 - library changes
- Forward to EWG/LEWG?

References

- [P3639R0] *Jan Schultke*. The `_BitInt` Debate (2025-02-20)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3639r0.html>
- [P3666R1] *Jan Schultke*. Bit-precise integers (2025-10-01)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3666r1.html>
- [N1744] *Michiel Salters*. Big Integer Library Proposal for C++0x (2005-01-13)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2005/n1744.pdf>
- [P1889R1] *Alexander Zaitsev et al.*. C++ Numerics Work In Progress (2019-12-27)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1889r1%2epdf>
- [P3639R0] *Jan Schultke*. The `_BitInt` Debate (2025-02-20)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3639r0.html>
- [N2763] *Aaron Ballman et al.*. Adding a Fundamental Type for N-bit integers (2021-06-21)
<https://open-std.org/JTC1/SC22/WG14/www/docs/n2763.pdf>
- [N2775] *Aaron Ballman, Melanie Blower*. Literal suffixes for bit-precise integers (2021-07-13)
<https://open-std.org/JTC1/SC22/WG14/www/docs/n2775.pdf>
- [N3699] *Robert C. Seacord*. Integer Sets, v3 (2025-09-02)
<https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3699.pdf>
- [N3705] *Phillip Klaus Krause*. bit-precise enum (2025-09-05)
<https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3705.htm>